Improving Efficiency in LLMs

JungHo Jung **Gene Lam** **Matt (Jiwoong) Park** **Norman Zhang**

1 Abstract

Advancements in large language models (LLMs) have revolutionized natural language processing applications, yet their resource-intensive nature poses significant barriers to accessibility and efficiency. Our project focuses on improving the efficiency of LLMs through Parameter-Efficient Fine-Tuning (PEFT) and Quantization techniques. Using SQuAD v2 dataset for extractive question answering, we developed a strong baseline with fine-tuned DistilBERT and demonstrate the impact of LoRA (Low-Rank Adaptation) on reducing trainable parameters by 99.98%. Despite initial challenges with accuracy and decoding efficiency, the work that we present sets the stage for scalable and efficient NLP systems for various real-world applications. Future directions of our work include applying quantization and exploring advanced LoRA variants for enhanced performance.

2 Introduction

Extractive question answering (QA) is a common and critical task in NLP where a system identifies an answer span from a given passage for a given query. QA involves understanding contextual relationships within the text and matching a query as a result to the most relevant information. This task is fundamental to many real-world applications, such as virtual assistants, chat bots, and educational tools. The complexity of training, testing, and deploying QA systems at scale makes it a perfect testbed for exploring resource-efficient methods in NLP. To illustrate the task of extractive question answering, consider the following example below:

Contextual Information:

"Beyoncé Giselle Knowles-Carter (/bijnse/ bee-YON-say) (born September 4, 1981) is an American singer, songwriter, record producer and actress. Born and raised in Houston, Texas, she performed in various singing and dancing competitions as a child, and rose to fame in the late 1990s as lead singer of RB girl-group Destiny's Child. Managed by her father, Mathew Knowles, the group became one of the world's best-selling girl groups of all time. Their hiatus saw the release of Beyoncé's debut album, Dangerously in Love (2003), which established her as a solo artist worldwide, earned five Grammy Awards and featured the Billboard Hot 100 number-one singles "Crazy in Love" and "Baby Boy."

Question:
"What album made her a worldwide known artist?"

Prediction:
"{ "text": ["Dangerously in Love"], "answer_start": [505] }"

In our example, the question-answer task requires identifying the correct span from the context based on the question, which is "Dangerously in Love."

Figure 1: Illustrative example of a question-answering task from the SQuAD v2 dataset.

The task of extractive QA is defined as follows:

Input: A text passage (context) X, consisting of n tokens $\{x_1, \ldots, x_n\}$, and a query Q, represented as a sequence of m tokens $\{q_1, \ldots, q_m\}$.
Objective: Train a model ϕ that predicts the probabilities of the start and end indices of the answer span, conditioned on the query Q and the context X:

$$\phi(\gamma_{\text{start}}, \gamma_{\text{end}} \mid Q, X).$$

Output: A span of tokens $\{x_{\text{start}}, \ldots, x_{\text{end}}\}$ in X, represented by indices $[start, end]$, such that $1 \leq \text{start} \leq \text{end} \leq n$.

The formulation above sums up the overall task, which is to locate the most relevant answer span within the passage by understanding the relationship between the query and the contextual information provided.

Beyond QA, our project also evaluates classification tasks using the GLUE benchmark, which is a collection of datasets designed to assess a model's understanding of natural language. The tasks we chose were sentiment analysis, textual entailment, and paraphrase detection. By considering both extractive QA and classification tasks, our goal is to evaluate how efficiency improvements in large language models (LLMs) generalize across different NLP tasks. We chose this combined focus because QA and classification tasks are central problems in NLP. Modern NLP systems, particularly LLMs, are highly resource-intensive, presenting a significant opportunity for optimizing computational efficiency. In this paper, we explore methods like Parameter-Efficient Fine-Tuning (PEFT), LoRA, and Quantization to address these challenges. Using benchmarks like SQuAD and GLUE, alongside metrics including Exact Match, F1, inference speed, and memory usage, we aim to demonstrate the practicality of these techniques in making LLMs more efficiency and accessible for real-world applications.

3 Literature Review

Low-Rank Adaptation (LoRA) is a method that addresses the challenges of fine-tuning large pre-trained language models through task-specific, dense modules. Hu et al. (2021), named *"LoRA: Low-Rank Adaptation of Large Language Models"*. The publication addresses the challenges of fine-tuning large pre-trained language models, such as GPT-3. GPT-3 includes 175 billion parameters and traditional fine-tuning methods require updating all model parameters, which can lead to extreme computational costs and memory demands. The authors of this paper propose a novel solution, Low-Rank Adaptation (LoRA), aimed to reduce the number of trainable parameters significantly by freezing the pre-trained weights of a model and introducing trainable low-rank matrices in specific model layers. LoRA achieves impressive results on common benchmarks like GLUE, WikiSQL, and SAMSum, showing that it can match or exceed the performance of traditional fine-tuning methods. Specifically, on the GLUE benchmark, LoRA proved to consistently perform within 1% of full fine-tuning accuracy while training fewer than 1% of total parameters. Furthermore, merging these updates into pre-trained weights during model inference has shown that LoRA avoids additional latency, making it highly scalable for real-world applications.

SmoothQuant, published by Xiao et al. (2023), addresses the challenges of quantizing LLMs to achieve both accuracy and efficiency without the need to retrain. Their research highlights challenges with quantizing activations in LLMs through smoothing the weights through per-channel scaling transformation without obstructing the saliency of outlier weights. Unlike partial quantization of LLM.int8, this allows full 8-bit quantization for both weights and activations. SmoothQuant maintains performance even with models containing billions of parameters, democratizing access to efficient LLM deployment. Although we do not directly use this paper's insights in our work, it helped deepen our understanding of post-training quantization, improving upon the LLM.int8 to further optimize the spatial complexity of large-scale models for real-world applications.

QLoRA, published by Dettmers et al. (2023), quantizes LoRA's transformer model to 4-bit precision while still maintaining a full 16-bit fine-tuning performance. The research uses three innovative techniques that allow this to happen. First is 4-bit NormalFloat (NF4), which takes a non-uniform distribution of quantized values of the base model's weights into 4-bits and freeze them during fine-tuning. Second is using Double Quantization, which quantizes the former quantization constants to reduce the average memory footprint. Lastly, QLoRA uses page optimizers to deal with memory spikes. It utilizes the CPU similar to a cache, where any out-of-memory pages from the GPU will be evicted to the CPU for later usage and new pages will be cached from the CPU to the GPU. QLoRA was a great choice of research and quantizer for this project because it is a pre-trained model that is customized for our fine-tuning model LoRA.

Adaptive Weight Quantization (AWQ) is a quantization method that offers a hardware-friendly approach for low-bit quantzation of LLMs (Lin et al., 2024). Unlike traditional techniques that treat all model weights equally, AWQ focuses on identifying and preserving only a minute fraction (0.1-1%) of the most critical weights called salient weights to analyze activation distributions rather than weight magnitudes. The weights that are associated with higher activation values are maintained at higher precision, and to reduce quantization error, AWQ employs a per-channel mechanism to scale salient weights by an optimized factor to balance performance and computational efficiency. This way, AWQ avoids mixed-precision setups and retraining, using data-driven scaling to minimize quantization errors. AWQ has been implemented and tested on LLMs like LLaMA and OPT with W4A16 quantization, achieving 3x faster speedup over FP16 implementations on GPUs and substantial gains on CPUs in real-world applications like TinyChat.

4 Experimental Design

4.1 Data

Our project utilizes publicly available datasets from the Hugging Face library that are designed specifically for extractive question answering and sequence classification tasks. The datasets include the Stanford Question Answering Dataset (SQuAD v2) for QA and the GLUE benchmark datasets (e.g., QNLI, MNLI, COLA, MRPC, and QQP) for classification tasks. The datasets mentioned were used to serve as the foundation for assessing the performance and efficiency of our quantization and Low-Rank Adaptation implementations.

SQuAD v2 was originally introduced as a widely used benchmark for question answering tasks (Rajpurkar et al., 2018). The dataset contains passages from Wikipedia paired with question and answer spans totaling to 130,319 training and 11,873 validation examples, with an average context length of 119.6 tokens (see Figure 2).

Most answer spans in the dataset are short and concise, with a majority of them ranging from 0 to 3 tokens. An important characteristic of the SQuAD v2 dataset is the inclusion of unanswerable questions, which account for roughly 33.4% of the training data (see Figure 3).

The percentage of unanswerable examples provides complexity to the task as models need to predict "no answer" when the question cannot be solved within the given context. To get a better understanding of SQuAD, we provide an specific context, question, and prediction example in Figure 4. In addition, GLUE benchmark datasets evaluate models on various natural language understanding tasks. The datasets were curated and preprocessed versions available in Hugging Face to streamline our implementation. Table 1 provides an overview of the dataset splits and labels. Overall, the skewed

Dataset	Train Set	Validation Set	Test Set	Inputs	Labels
SQuAD v2	130,319	11,873	N/A	Context/Question	Start/End positions of answer
QNLI	104,743	5,463	5,463	Sentence	Entailment/Non-Entailment
MNLI	392,702	9,815	9,796	Premise/Hypothesis	Entailment/Contradiction/Neutral
COLA	8,551	1,043	1,063	Sentence	Label
MRPC	3,668	408	1,725	Question1/Question2	Equivalent/Not-Equivalent
QQP	363,846	40,430	390,965	Question1/Question2	Duplicate/Not Duplicate

Table 1: Dataset details for each of the experiments we ran.

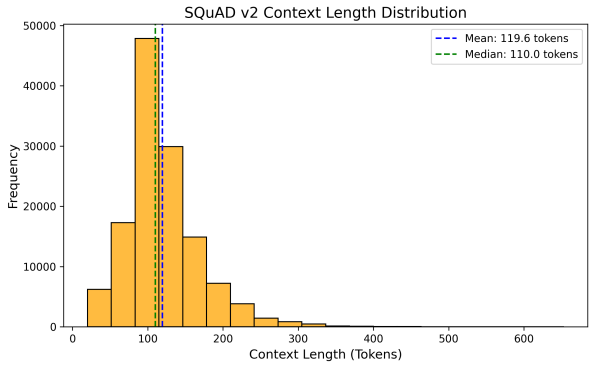


Figure 2: Histogram of context lengths with mean and median annotations in the SQuAD v2 dataset.

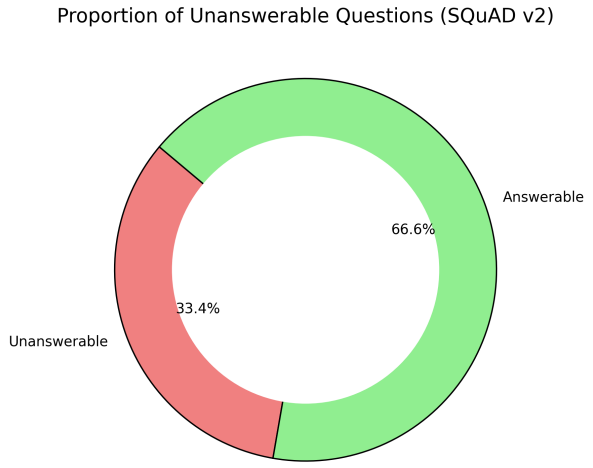


Figure 3: Proportion of unanswerable vs. answerable questions in the SQuAD v2 dataset.

distributions in datasets like SQuAD v2 poses unique challenges for models, making it well-suited for evaluating our project methods of improving model efficiency.

4.2 Evaluation Metric

We evaluated our QA system using Exact Match (EM) and F1 score, which are two standard metrics in QA benchmarks like SQuAD (Le et al., 2024). Exact Match measures the proportion of predictions that match the ground truth answer. F1 score utilizes both precision and recall to evaluate partially correct answers. Precision includes the proportion of predicted tokens that are accurately identified, while recall is the proportion of ground truth tokens captured in the predictions of our model. We use the following formulas to evaluate performance:

$$EM = \frac{\text{Number of Exact Matches}}{\text{Total Number of Questions}} \tag{1}$$

$$F_1 = 2 * \frac{Precision * Recall}{Precision + Recall} \tag{2}$$

where

$$Precision = \frac{\#\text{ Correct Tokens in Prediction}}{\#\text{ Tokens in Prediction}} \tag{3}$$

and

$$Recall = \frac{\#\text{ Correct Tokens in Prediction}}{\#\text{ Tokens in Ground Truth}} \tag{4}$$

In addition to traditional QA metrics, we also evaluated our model efficiency by incorporating metrics relevant to LoRA and quantization. Training time was another metric we used to record the speedup achieved through the use of LoRA adapters compared to traditional fine-tuning. Another measure is the number of trainable parameters relative to the total parameters in the model. LoRA significantly reduces the number of trainable parameters by freezing pre-trained weights and introducing low-rank matrices. For quantization, we also evaluated the impact of bit precision, such as 8-bit versus 16-bit on memory usage and inference latency. These metrics provide a thorough assessment by evaluating how effectively LoRA and PEFT maintain model performance while improving efficiency. Their applicability is demonstrated in relevant works such as those by Prabhakar et al. (2024) and Le et al. (2024).

4.3 Simple Baseline

To evaluate the difficulty of tasks in our project, we established a simple baseline for both extractive QA and classification systems. For the SQuAD v2 dataset, a

baseline includes the approximate 33.4% of unanswerable questions in the training data. This is because if the model were to predict no answer for every query, it would achieve an Exact Match (EM) score of 33.4%. This baseline serves as a minimal performance threshold rather than a comprehensive evaluation of model capabilities. By comparing our model performance generated against these benchmarks, we convey how our approaches enhance both efficiency and accuracy, particularly in tasks that require deeper understand and adaptability to imbalanced data.

5 Experimental Results

5.1 Published Baseline

The experiments we conducted were similar to that of Hu et al. (2021), but modified for QA tasks. As a result, our results are not directly comparable but we will use their results as a proxy for evaluating our results. The table below presents a selection of results comparing a fine-tuned model and a LoRA-adapted model, evaluated on the GLUE benchmark.

Model	FT or LoRA	Trainable Params	GLUE Avg (%)
RoBERTa base	FT	125 M	86.4
RoBERTa base	LoRA	0.3 M	87.2
DeBERTa xxl	FT	3 M	88.4
DeBERTa xxl	LoRA	0.8 M	88.6

Table 2: Comparison of FT and LoRA for RoBERTa base and DeBERTa xxl on GLUE.

Based on their results, we should expect to see drastic decreases in trainable parameters while maintaining or surpassing baseline performance.

We first fine-tuned our DistilBERT base model on the SQuAD v2 dataset for two epochs and evaluated the performance using Exact Match and F1 compute metrics. Using an inverse square root schedule with a starting learning rate of $\alpha = 1e-5$, the training loss steadily decreased across steps, as demonstrated in Figure 4.



Figure 4: The training loss on the published baseline decreases steadily with each step. We used an inverse square root scheduler with a starting learning rate of $\alpha = 1e-5$. All experiments are run on a g5.xlarge AWS instance.

Our benchmarks are recorded in the table below:

Metric	Score
EM	48.87
F1	15.14
Training time	44m 51s
Trainable parameters	66364418

We observed that our EM score indicates a reasonable ability to correctly predict the exact answer span while F1 score highlighting challenges in partially correct predictions. Our results suggest that the model often predicts either completely correct or incorrect spans without adequately capturing partial overlaps. The gap between EM and F1 score can be partially attributed to our decoder's current approach, which uses argmax of start and end logits without additional processing. This simplified decoding method includes unnecessary context or truncates critical parts of the answer span, as shown in the example below:

Predicted: "denmark, iceland and norway who, under their leader rollo, agreed to swear fealty to king charles iii of west francia. through generations of"

Answer: "denmark, iceland and norway"

In previous iterations of our decoder, we were able to gain better performance by iterating through the top logits of the model output and returning the top x answer spans. However, this was computationally inefficient for a deep learning network. Here, we just take the argmax of our start and end logits. Another challenge was with the tokenizer's truncation and formatting of long sequences. Each model architecture has a maximal token length it can process (we used DistilBERT's max length of 384). If a sequence goes beyond that, the tokenizer spills the leftover sequence to the next sample and updates the overflow map. However, this led to issues with uneven arrays when batching despite adding padding. We mitigated this by not using the overflow map in our decoding step, removing critical context indices. Our subsequent experiments with LoRA aim to overcome these challenges while improving both efficiency and performance.

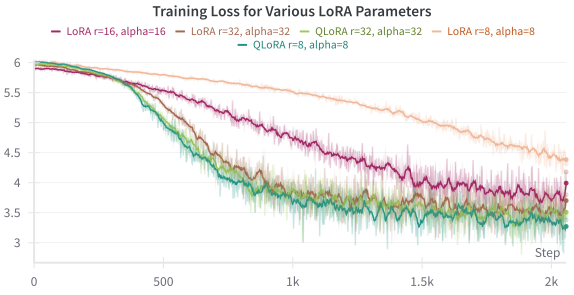


Figure 5

Table 3: Comparison of different LoRA and QLoRA configurations.

Method	Trainable Parameters	Exact Match (%)	F1 (%)	Training Time (min)
No LoRA	66,364,418	48.87	15.14	44m 51s
LoRA ($r = 8, \alpha = 8$)	222,722	50.98	0	17m 31s
QLoRA ($r = 8, \alpha = 8$)	224,260	50.98	0	40m 58s
LoRA ($r = 16, \alpha = 16$)	443,906	50.98	0	17m 53s
LoRA ($r = 32, \alpha = 32$)	886,274	50.98	0	17m 50s
QLoRA ($r = 32, \alpha = 32$)	887,812	50.98	0	41m 13s

Method	# train params	CoLA	MRPC	QNLI
Full-FT	124,647,170	57.1	90.4/86.9	90.9
LoRA	960,770 (0.7%)	44.4	86.7/81.6	88.7

Table 4: GLUE subset. CoLA is evaluated using Matthews Correlation; MRPC is evaluated using F1/accuracy; QNLI is evaluated using accuracy

	Baseline	LoRA	QLoRA
Model Size	253.164MB	253.164MB	**65.834MB**

Table 5: Difference in model size in baseline, LoRA, and QLoRA. QLoRA shows clear improvement in spatial complexity.

5.2 Extensions and Error Analysis

As part of the extensions, we experimented with different LoRA matrix dimensions. For example, we experimented with adding more LoRA layers in parallel. However, they yielded poorer results. In order to dive deeper into how the specific configurations of LoRA affect the model performance, we conducted an extensive comparison with various parameters on the classification datasets, CoLA, MRPC, and QNLI. We tested on these hyperparameters: rank r, which denotes the dimensionality of the LoRA module, and the scaling constant α, which adjusts the magnitude of the LoRA weight updates. All sweeps were trained on 15 epochs with varying r and α.

The Corpus of Linguistic Acceptability (CoLA) dataset is used for evaluating whether a given sentence is grammatically acceptable in English. This is a challenging task, requiring the model to understand complex linguistic patterns. The best configuration for our CoLA sweep was $\alpha = 10, r = 10$. Having a high value for both enabled it to capture grammatical and linguistic rules.

The best configuration for Microsoft Research Paraphrase Corpus (MRPC) had an α scale of 10 and an r scale of 6. The MRPC is a binary classification task to determine whether two sentences are paraphrases of each other. This task requires the model between fine-grained semantic relationships between sentences. We believed the high α term boosted the fine-tuned weights learned from the LoRA adapter and enabled it to capture these semantic nuances.

Lastly, Question-answering Natural Language Inference (QNLI) is another binary classification task that answers whether a given sentence answers a specific question. We found that the best parameters for the LoRA adapter was a low $\alpha = 6, r = 6$. This tells us that a smaller module was sufficiently expressive for the task of entailment compared to the CoLA and MRPC datasets.

6 Conclusions

For our project, we implemented techniques like Parameter-Efficient Fine-Tuning (LoRA) and quantization to improve the efficiency of large language models across question answering and classification tasks. Using datasets such as SQuAD v2 and GLUE standard benchmarks, we aimed to reduce computational overhead while maintaining model performance. LoRA helped enable us to fine-tune models by updating a fraction of their parameters, significantly lowering memory and computational requirements and speeding up training time. A combinations of LoRA and quantization methods also reduced model size by converting model weights to lower bit precision, but we didn't observe speed ups in training time. Since our implementation did not produce the results we wanted, we expanded our benchmarking on the sequence classification and natural language inference on a subset of the GLUE dataset.

Our results highlight the trade-offs between efficiency and performance. Our project demonstrated the impact and importance of using LoRA and quantization to make LLMs more efficient and accessible, especially in resource-constrained environments.

7 Acknowledgments

We would like to thank our TA Aditya Jayant Ganapathiraju for providing invaluable assistance throughout our final project. His guidance in clarifying key questions and helping us understand the requirements greatly contributed to the success of our project. Additionally, we would like to acknowledge the authors and developers of the Hugging Face Transformers library. Their codebase served as a foundation for many of our experiments and implementations.

References

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Thanh-Dung Le, Ti Ti Nguyen, and Vu Nguyen Ha. 2024. The impact of lora adapters for llms on clinical nlp classification under data limitations.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*.

Akshara Prabhakar, Yuanzhi Li, Karthik Narasimhan, Sham Kakade, Eran Malach, and Samy Jelassi. 2024. Lora soups: Merging loras for practical skill composition tasks.

Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know what you don't know: Unanswerable questions for squad. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*.